

Instructions for End-Semester Digital Communication Lab

1 Convert video to bits

Convert roll_number.mp4 into a binary array. Since MP4 is already a source-coded (compressed) format, no separate source coding is required. Use the bits directly from the encoded file.

2 Make chunks

You cannot reliably transmit the entire bitstream in one transmission because frequency offset and synchronization errors vary across long transmissions. Your measured frequency offset is not uniform over entire received array. Hence, send the data in chunks.

3 Chunk size (default)

Split the bitstream into chunks of **8000 bits** each. You may increase this after experiments to find the maximum error-free chunk size.

4 Chunk ID

To each 8000-bit chunk append an **8-bit chunk_ID** (e.g., ID=1 → 00000001, ID=15 → 00001111). Resulting length before CRC = **8008 bits**.

5 CRC

Compute and append CRC bits (suggestion: **16-bit CRC**) to each (so $8000 + 8 + 16 = \mathbf{8024 \text{ bits}}$ per chunk with the example values).

6 Channel coding

Apply a channel code (choose one from Hamming, Convolutional, Turbo, BCH, LDPC, or Polar). Use a **coding rate (k/n)** approximately **0.5–0.7**. Note: Hamming corrects only single-bit errors; stronger codes (LDPC, Polar, BCH, Turbo) can correct more for similar rates. Example: rate 0.5 → coded length = **16048 bits**.

7 Modulation

Map coded bits to symbols.

- BPSK — 1 bit/symbol → 16048 symbols.
- 4-QAM — 2 bits/symbol → 8024 symbols.
- Higher orders (8-PSK, 16-QAM) possible but need more SNR. Fewer symbols → better frequency synchronization; choose modulation accordingly. (Classroom has 4-QAM end-to-end code.)

8 Pre-transmit processing

Append a Barker sequence (pilot) and apply pulse-shaping (e.g., raised-cosine) before SDR transmission.

9 Transmit duration per chunk

Transmit each chunk for **t = 10 s** by default. You may reduce t after measuring CRC failure probability.

10 Receiver capture

At the receiver, capture a large buffer of raw samples (e.g., ~500000 samples) and perform matched filtering, synchronization, frame detection, phase compensation, etc., to extract the modulated symbols for that chunk.

11 Demodulate & decode

Demodulate symbols → obtain channel-coded bits → perform channel decoding → recover the 8024 bits (in example). Then verify the 16-bit CRC.

- If CRC fails: discard this reception of the chunk and wait for the next transmission.
- If CRC passes: extract chunk_ID and the 8000 payload bits.

12 Duplicate handling

During the transmit interval you may successfully receive the same chunk multiple times. For each chunk_ID keep the 8000 payload only once in the collector (ignore subsequent duplicates after a CRC-pass for that ID).

13 Timing / sequencing

After the transmit interval for one chunk (t seconds) the transmitter moves to the **next chunk**. Implement the whole process in loops so everything runs automatically (TX side cycles chunks; RX side listens, decodes, and stores unique chunks).

14 Reconstruction

When all chunks are collected (in order) and verified, concatenate the 8000-bit payloads to reconstruct the original bitstream and write back to roll_number.mp4.

15 Worked example

If video size = **8 KB = 64,000 bits**, with 8000-bit chunks you need 8 chunks $\rightarrow$ at $t = 10$ s each $\rightarrow$ **80 s** total (ignoring processing overhead).

16 Experiment & optimize

Measure and tune: chunk size, CRC length, channel code & rate (k/n), modulation order, transmit time t , receiver processing/latency, and possible parallel decoding at receiver. Vary them based on CRC failure rate, throughput, and synchronization reliability.